CODERS IN PUNE · KNOWLEDGE HANDOVER CHALLENGE · ROUND 2: THE BUILD

Baton

The plan is now a **working product.** Evidence-linked, permission-aware handovers that a judge can open and use, with no walkthrough needed.

Live product
baton-handover-sigma.vercel.app
No sign-up. Click "Try the live demo".

Source code
github.com/SidharthMonangi/Codersinpune-branch *round2-build*, folder *app/*

Measured, not claimed
10/10 automated tests · 0 of 20 permission checks leaked · full judge journey verified on the live site

Team **leisure** · Solo entry by **Sidharth Monangi**

Submitted **November 2026**

SUBMISSION AT A GLANCE

Round 2 summary

Required item	Where to find it
Working project link	**https://baton-handover-sigma.vercel.app**. It opens without sign-in; each visitor gets a private workspace.
Source-code repository	**https://github.com/SidharthMonangi/Codersinpune-** (branch *round2-build*; app in *app/*; README at the root)
Completed features	§1 (all 10 MVP features from the Round 1 plan, plus 4 extras)
Technology and implementation	§3 architecture and data flow, §4 implementation details, APIs, models, datasets and libraries
Changes to the original plan	§5, with the reason for each change
Testing evidence	§6: automated tests, end-to-end browser runs on the live site, mobile and security checks
Three Round 3 improvements	§7

The 2-minute judge path

1. Open the link and click **Try the live demo → Open the Project Orion demo**. This is a synthetic student-club project: Priya (tech lead) leaves on 30 Nov and Arjun takes over.
2. Reconstruct & verify: click any grey citation chip to see the exact source, with shared words highlighted. Open **Conflicts** to see reversed decisions and unresolved disagreements.
3. Gap Interview: set *Acting as* to **Priya Nair (leaver)** and answer one question. It becomes ATTESTED evidence and the Readiness Score rises.

4. Review & approve: switch to **Dr. Anjali Kulkarni (approver)**, accept or edit claims, then approve. The version is hashed and frozen.
5. Handover: compare **Arjun (Tech Lead)** and **Rohan (Volunteer)** side by side; Rohan sees 🔒 sealed placeholders. Create a link and open it in a private window to see the recipient's plan, unknowns, people to talk to, and acknowledgements.
Bring your own data: start an empty workspace, then upload exports or connect any public GitHub repo. Sample files are on the Connect tab.

Measured on the demo dataset (synthetic, with a planted answer key)

6/6
planted decisions extracted, each with citations

3/3
reversed decisions marked superseded (MongoDB→Postgres, OneSignal→email, weekly→biweekly)

2/2
factual contradictions flagged (fest date 14 vs 15 Dec, registration cap 800 vs 1000)

0/20
confidential-item leaks across 20 role × item checks (a test failure blocks the build)

5/5
planted gaps raised as Gap Interview questions (webhook owner, migration 0009, SSO certificate, Razorpay contact, sole-knower areas)

10/10
automated tests passing (leakage, redaction, conflicts, gaps, attestation access, citations, AI guard-rails, fallback)

38 s
full judge journey run by a browser robot on the live site: 0 errors, 0 leaked strings in the volunteer view

49 ms
to rebuild the project state from 11 sources and 96 evidence items (rule engine)

§1 · COMPLETED FEATURES

Every MVP feature from the Round 1 plan is working

#	Feature	What works now	Status
F1	**Handover workspace**	A private workspace per visitor (secure cookie, no sign-up). Setup for the departing member, last day, recipients and roles, and people. An "Acting as" switch lets one tester play admin, leaver and approver.	**Done**
F2	**Source ingestion**	Parsers for Slack export JSON, Discord export JSON, GitHub export JSON, Markdown/TXT docs, meeting transcripts (.vtt, speaker-aware) and Trello/Jira CSVs. A live, read-only GitHub connector for any public repo (issues, PRs, commits). Each source has an access list and a sensitivity level.	**Done**
F3	**Redaction gate**	Removes API keys and tokens, passwords, JWTs, phone numbers, Aadhaar, PAN, card numbers and high-entropy strings *before* storage or any model call. Automatic sensitivity escalation for money, contracts and health.	**Done**
F4	**Evidence Ledger**	8 claim types, each linked to exact evidence items. Duplicates are merged; recurring problems are collapsed with a report count; a later decision on the same option slot supersedes the earlier one.	**Done**
F5	**Verifier & trust states**	Each evidence link is checked against the statement it was extracted as. Trust states: Verified (2+ sources or an authoritative one), Inferred, Conflicting, Attested. Unsupported claims are dropped.	**Done**
F6	**Timeline, ownership & conflicts**	Decision timeline including reversals, a who-owns-what map with unowned work highlighted, an area summary, and a side-by-side Conflicts view.	**Done**
F7	**Gap detector + Gap Interview**	Six gap kinds, ranked by risk using the Sole-Knower Index and urgency, capped at 15. Answer, Don't know, route to someone else, or skip. Answers become ATTESTED evidence that inherits the strictest access of what they answer.	**Done**
F8	**Review screen**	Accept, edit, reject or undo per claim; bulk-accept verified claims; approve as a named approver; version history with SHA-256 hash and diff counts; append-only audit log.	**Done**
F9	**Role-based handover + Sealed Claims**	Built on the server per recipient from the approved version. Day 1 / Week 1 / Month 1 plan, people to talk to, known unknowns, sealed placeholders grouped by area with who to ask, and a flag on work still owned by the leaver. Unguessable share links, acknowledgement of receipt and of each critical item, side-by-side role comparison.	**Done**
F10	**Readiness Score**	A live 0–100 score in the header, using the published formula (risk-weighted gaps closed + claims reviewed).	**Done**

Built beyond the plan

Team knowledge radar. The Sole-Knower Index for *every* member, per area, on the Overview. It shows where knowledge would be lost if any one person left today, so knowledge is captured while the work is happening, not only at resignation.

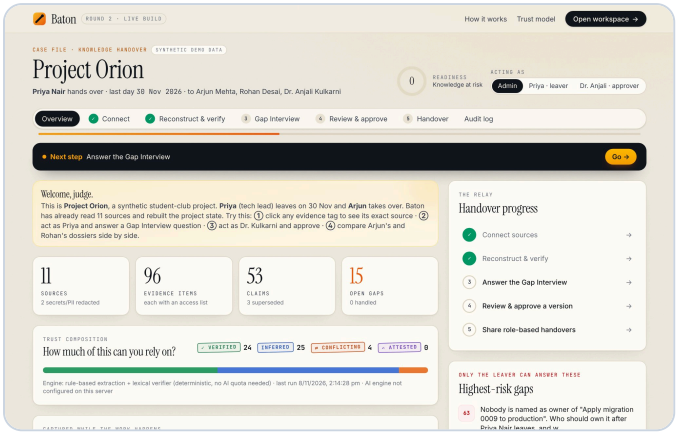
Per-reader re-rendering. If a claim's main wording came from a source the reader can't open, Baton shows the wording extracted from the reader's own permitted source instead, or seals the claim.

AI engine with guard-rails (code-complete). Gemini extraction with JSON-schema output, plus a separate per-link AI verifier. Claims citing non-existent evidence are dropped. On errors or rate limits it falls back to the rule engine and says so in the UI.

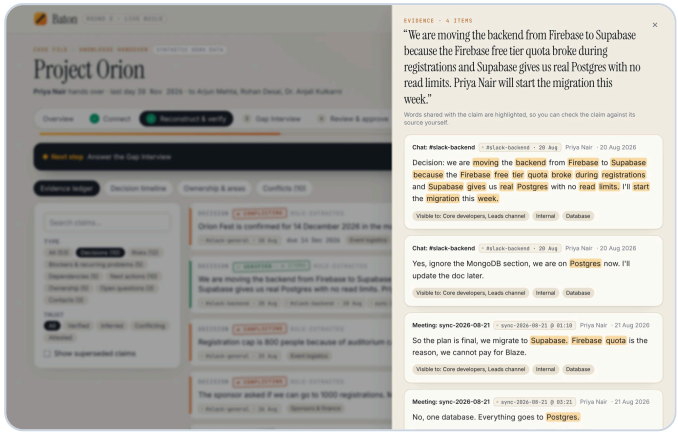
Guided experience for judges. A "Next step" banner, a progress checklist, an inline trust legend, sample files to download, and clear empty states and error messages.

§2 · THE PRODUCT, SCREEN BY SCREEN

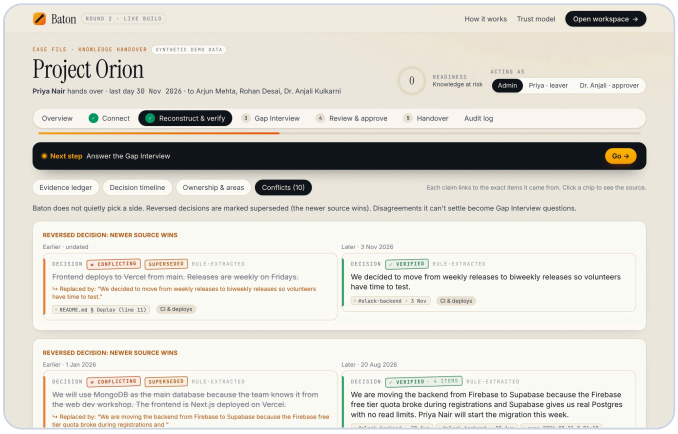
From connected sources to an acknowledged handover



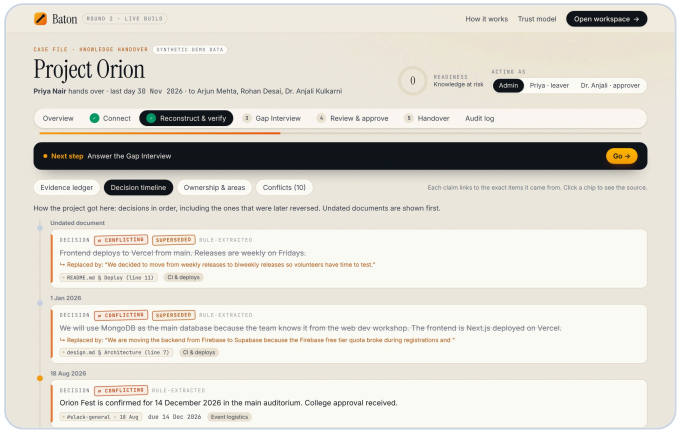
Overview. Counts by trust state, the progress checklist, the highest-risk gaps, and the team knowledge radar (Sole-Knower Index per person × area).



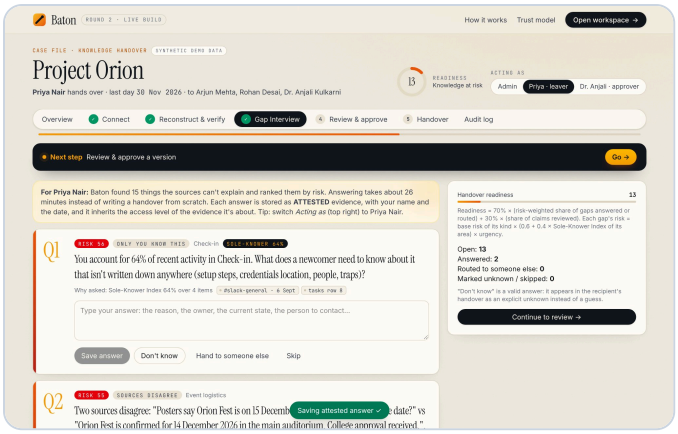
Evidence drawer. Every claim opens its exact sources, with shared words highlighted, author, date, access list and sensitivity.



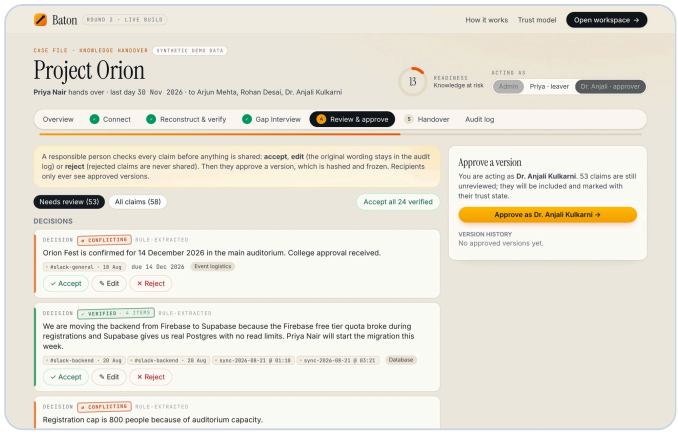
Conflicts. Reversed decisions are marked superseded (the newer source wins); unresolved disagreements go to the Gap Interview.



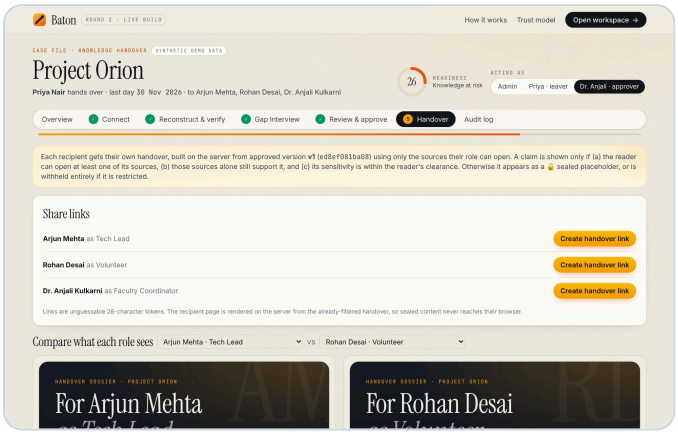
Decision timeline. How the project got here, including the decisions that were later reversed.



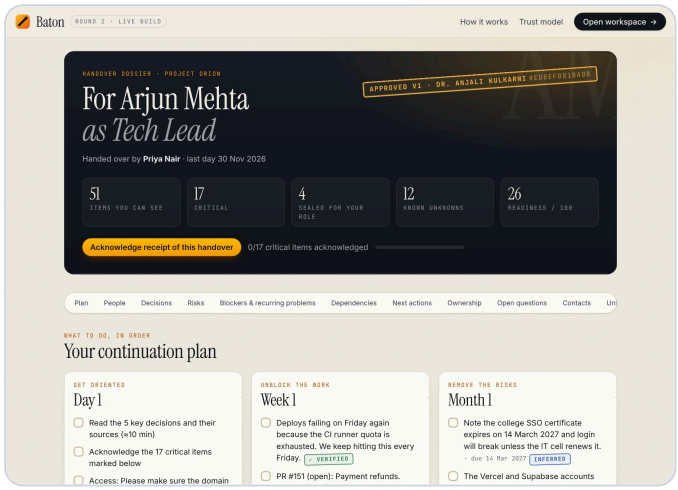
Gap Interview. Questions ranked by risk, each with why it was asked and its evidence. Answers become ATTESTED evidence and Readiness rises.



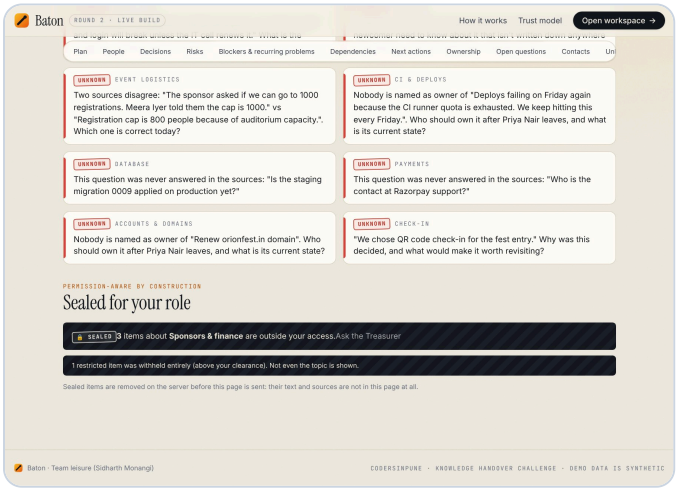
Review & approve. Accept, edit or reject per claim, then approve as a named person. Versions are hashed and frozen.



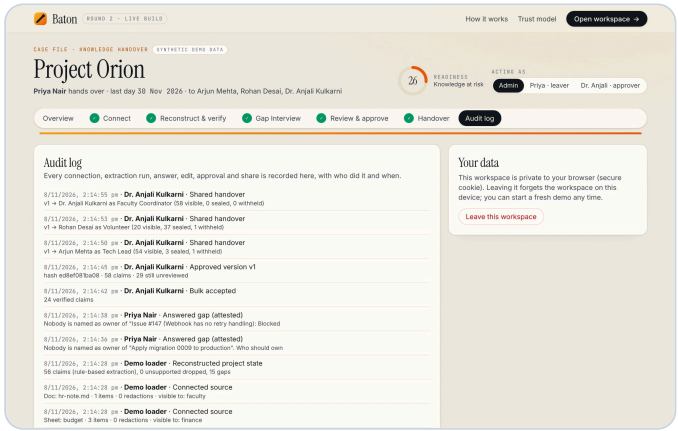
Role comparison. Same approved version: the Tech Lead sees 52 items; the Volunteer sees 18, plus sealed placeholders.



Recipient view. Continuation plan, people to talk to, acknowledgements, and a jump navigation.



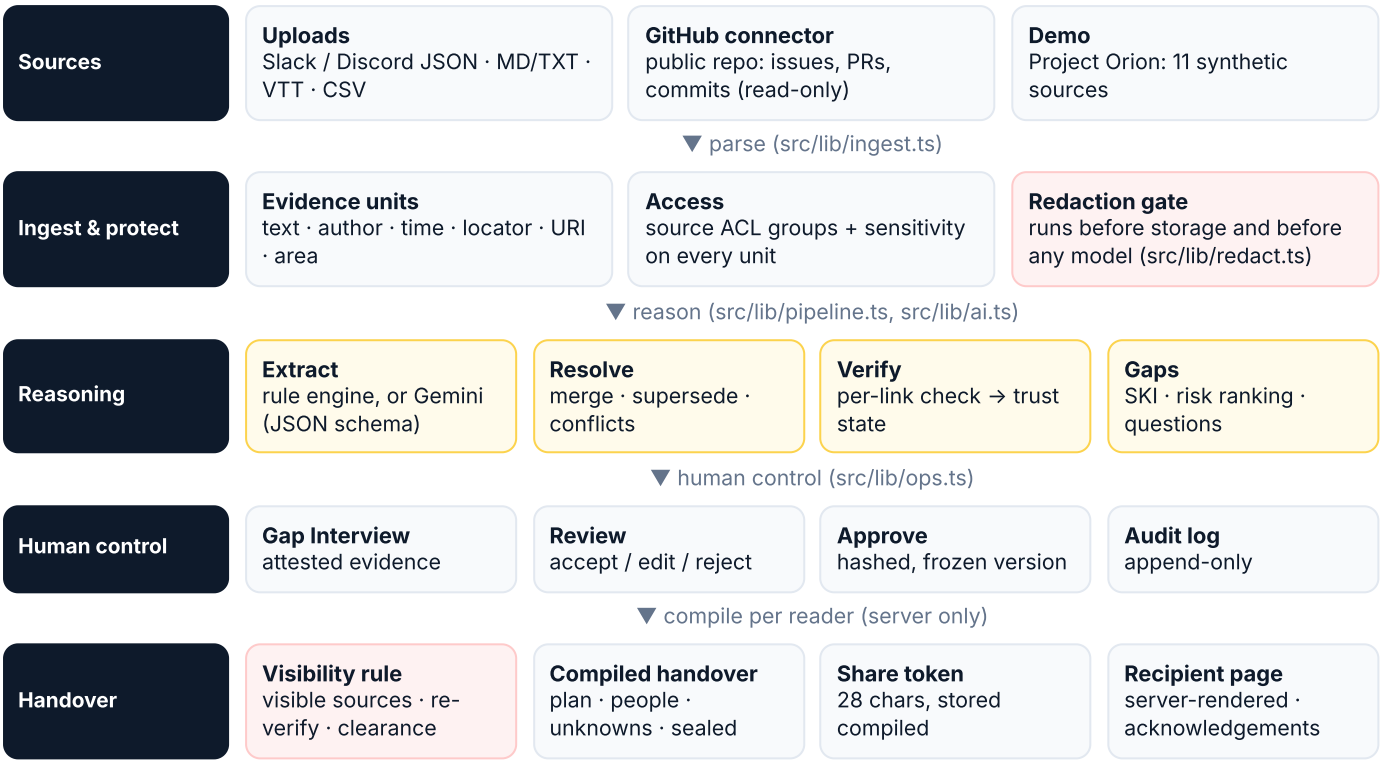
Known unknowns and sealed items. Baton lists what it doesn't know instead of guessing; sealed content is removed on the server.



Audit log. Every connection, run, answer, edit, approval and share, with who did it and when.

§3 · ARCHITECTURE AND DATA FLOW

How a source item becomes a line in someone's handover



Key technical decisions

Decision	Why
Compile every handover **on the server**, per reader, and store only the compiled result behind the share token	Sealed content never reaches the reader's browser. Permissions are enforced on the output, not just on retrieval.
Storage through **database functions only** (Supabase RLS with no policies + SECURITY DEFINER RPCs)	The public key can't read or write any table directly. Every call needs the workspace secret (stored as a SHA-256 hash) or a share token. Tested: a wrong secret returns null; a direct table read returns *permission denied*.
Two extraction engines behind one pipeline	The rule engine is deterministic, free and instant, and it keeps the demo working with no AI quota. Gemini adds better paraphrasing when a key is configured. Both go through the same resolver, verifier, trust rules and permission checks.
Verify each evidence link separately	This lets Baton re-check a claim against *only* the sources a given reader can open, which is the core of Sealed Claims.
A synthetic dataset **with a planted answer key**	The brief requires synthetic data for demos. An answer key turns "it seems to work" into measurable recall and leakage tests that run in CI.
Workspace per visitor, **no sign-up**	Judges must be able to use the product without assistance. Each judge gets an isolated copy of the demo, so testers can't interfere with each other.

§4 · IMPLEMENTATION DETAILS

Models, rules and the permission logic

The visibility rule (src/lib/ops.ts)

```
E_r = {links of c whose evidence r can open (ACL ∩
groups ≠ ∅ and sensitivity ≤ clearance)}
visible(c, r) ⇔ ∃ supporting link in E_r
  ∧ c's wording is supported by E_r alone (else re-
render from E_r's own statement)
  ∧ sensitivity(text) ≤ clearance(r)
otherwise → SEALED (or withheld if Restricted)
```

Gap questions are shown to a reader only if *all* of their linked claims are visible to that reader, because a question can leak content too.

Scores

```
SKI(area) = Σ w(e)·[author=leaver] / Σ w(e), w =
e^-age/90d
gap risk = base(kind) × (0.6 + 0.4·SKI) × urgency
Readiness = 100 × (0.7 × risk-weighted gaps closed +
0.3 × claims reviewed)
```

Trust states

Verified: a supporting link scores ≥ 0.5 and comes from 2+ sources, or from an authoritative source (repo, doc, task board, attestation), or is an explicit decision. **Inferred**: one informal source. **Conflicting**: an unresolved contradiction, or superseded. **Attested**: Gap Interview answer. Claims with no support are dropped.

APIs, models, datasets, libraries, services

Component	Use
Next.js 15 · React 19 · TypeScript	Web app, API routes, server-rendered recipient pages. Custom "handover dossier" design system (Instrument Serif, Inter, JetBrains Mono, self-hosted)
Tailwind CSS 4	UI styling (no component template used)
Supabase Postgres	Storage: RLS + 7 RPC functions, pgcrypto for secret hashing (Mumbai region)
Vercel	Hosting (Mumbai region), production deploys from GitHub
GitHub REST API	Live read-only repository connector
Google Gemini API (gemini-2.5-flash, free tier)	Optional AI extraction and per-link verification, with JSON-schema output. Enabled by setting *GEMINI_API_KEY*
Project Orion dataset	Synthetic, written for this project: 11 files, 96 evidence items, planted answer key
Node test runner · tsx · Playwright	Unit/integration tests; browser end-to-end runs
GitHub Actions	CI: tests, then type-check, lint and build on every push
Claude (AI coding assistant)	Used during development, as permitted and disclosed

Prompt-injection defence

Source text is passed to the model as quoted JSON data with an explicit "data, not instructions" system rule. The model has no tools, and its output must match a schema and cite real evidence IDs. Visibility is decided by code and the database, never by the model.

§5 · CHANGES FROM THE ROUND 1 PLAN

What changed, and why

Round 1 plan	Round 2 build	Reason
Next.js front end + FastAPI (Python) back end + Redis workers	**One Next.js app** (UI + API routes)	A solo builder with 72 hours: one language, one deploy, half the moving parts. Ingestion of the demo project takes 49 ms, so no queue is needed yet.
Claude API for extraction and verification	**Gemini free tier** (optional) + **rule engine**	Zero cost during judging. The rule engine keeps the product working with no key or when the free-tier quota runs out, and it is honestly labelled in the UI.
Microsoft Presidio for PII	**Regex + entropy redaction gate**	Presidio needs a Python service. The gate covers the PII and secret types relevant to Indian student teams (phone, Aadhaar, PAN, cards, API keys, passwords) and is unit-tested.
Sign-in with Google/GitHub OAuth	**Private workspace per visitor** + unguessable share links	The Round 2 rules require judges to use the product without sign-in. Real identity moves to Round 3 (§7).
pgvector embeddings for de-duplication	**Token-overlap and option-slot matching**	Deterministic and explainable at this data size. Embeddings return with the AI engine at scale.
Render/Railway for the API	**Vercel only**, functions in Mumbai	Co-located with the Supabase database (ap-south-1) for low latency.
Team knowledge view as future scope	**Built now** (Team knowledge radar)	The organisers' Round 2 brief stresses capturing knowledge "while the work is happening". The Sole-Knower Index for every member does that.

The core workflow, evidence model, trust states, Gap Interview, Sealed Claims rule, Readiness formula and demo dataset are exactly as specified in Round 1.

§6 · TESTING EVIDENCE

Tested like it has to be trusted

Automated tests (app/tests, run in CI on every push): 10/10 passing

Test	What it proves
Confidential items never reach roles without clearance	Sponsorship amount, welfare note, password and phone number stay out of every unauthorised role's compiled handover (20 role × item checks)
Cleared roles still see what they're allowed to see	The permission system isn't simply hiding everything
Secrets and PII are redacted before storage	No planted secret appears in any stored evidence
Planted decisions, reversals and conflicts are detected	6/6 decisions, 3/3 supersessions, 2/2 contradictions
Unowned work and sole-knower areas become questions	5/5 planted gaps appear in the Gap Interview
Attested answers inherit the strictest access	An answer can't leak what it answers about
Parsers reject bad files with clear messages	Unsupported types and broken JSON fail safely
Every shared claim has a citation the reader can open	No uncited statement ever reaches a recipient
AI path: invented citations are dropped; AI claims respect access	Stubbed Gemini returns a fake evidence ID and a leads-only fact; the first is discarded and the second never reaches the volunteer
AI failure falls back to the rule engine and says so	A 429 rate limit still yields a full ledger, with a visible note

End-to-end on the live site

A Playwright robot ran the full judge journey on the production URL: demo → evidence drawer → conflicts → two answers and one "don't know" as Priya → bulk-accept and approve as Dr. Kulkarni → three share links → acknowledgements → volunteer link opened in a fresh browser with no cookies. **38 seconds, 0 console errors, 0 leaked strings.** A second run tested the bring-your-own-data path on the live site: an empty workspace, the public repo *vercel/ms* through the GitHub connector, two uploaded files, then reconstruct and approve. Result: 30 cited claims and 14 Gap Interview questions, with no errors.

Security and device checks

Wrong workspace secret → *null*; direct table read with the public key → *permission denied*; GitHub push protection caught a fake key in an early version of the demo data, which was replaced. iPhone 13 emulation: no horizontal scroll on any tab after a layout fix.

Known limitations (stated honestly)

- The rule engine catches clearly worded statements; implicit knowledge is missed or labelled Inferred. The AI engine improves recall but is limited by the free-tier rate.
- Topic areas (used for the Sole-Knower Index) come from a keyword classifier tuned on the demo domain. On unfamiliar repositories they are coarser until the AI engine is enabled.
- Access lists for uploaded files are set by the uploader. Mirroring permissions automatically from the source system is Round 3 work (§7).
- Workspaces are tied to one browser (cookie). Anyone holding a share link can open that one compiled handover, which never contains sealed content.
- Precision has been spot-checked by hand but not formally measured yet. A labelled precision benchmark is part of Round 3.

§7 · THREE MAJOR IMPROVEMENTS COMMITTED FOR ROUND 3

From a one-off handover to always-ready continuity

1 Ask Baton: evidence-grounded Q&A for the incoming person

After the handover, the newcomer asks questions in plain language ("why is the webhook not idempotent?"). Answers are generated **only** from the approved handover and the sources that reader is permitted to see, with citations on every sentence. When the evidence isn't there, Baton says *"not in evidence"* and routes the question to the most likely owner. Their reply comes back as attested knowledge, so the handover keeps improving after the leaver has gone.
Done when: answers carry citations; there are zero answers from outside the reader's permitted sources (CI test); and the "not in evidence" rate on a 30-question test set is reported.

2 Living handover: live connectors, continuous capture and stale-handover alerts

Live Slack/Discord and GitHub connections (OAuth + webhooks) with incremental sync, so the ledger updates **while the work happens**. A drift detector compares new activity with the last approved version and alerts the owner, for example: "v3 is stale: 4 claims changed, 1 new conflict". Each person's readiness is then always current, before anyone resigns.
Done when: a new message or PR appears in the ledger within a minute; a planted reversal triggers a stale alert; and re-approval produces a version diff.

3 Verified identity and mirrored permissions

Sign-in with Google or GitHub for admins, leavers, approvers and recipients. Share links additionally require the recipient's verified identity. Access lists are **imported automatically** from the source systems (GitHub collaborators and teams, Slack channel membership) instead of being set by the uploader, and are re-checked at compile time. Combined with the AI engine, this closes the main gap between the demo and real organisational use.
Done when: a recipient can't open someone else's link; a user removed from a Slack channel loses those claims on the next compile (CI test); and an admin can see who accessed what.

These are product capabilities, not cosmetic changes. Each is measurable, and each extends one of the three success criteria in the brief: traceable (1), actionable (2), permission-aware (3).

TEAM AND DISCLOSURE

Team leisure

Field	Details
Team	**leisure** (solo entry)
Participant	**Sidharth Monangi**: product, AI pipeline, back end and security, front end, testing, deployment, documentation
Email	sidharthmonangi@gmail.com
Live product	https://baton-handover-sigma.vercel.app
Repository	https://github.com/SidharthMonangi/Codersinpune- (branch round2-build)

Disclosure

All product code was written during the official Round 2 build window (from 8 Nov 2026, 10:00 IST); the commit history shows this. No templates or boilerplate were used beyond the standard *create-next-app* scaffold. AI assistance (Claude) was used for coding and drafting, as the rules permit. All demo data is synthetic and was written for this project; any resemblance to real people or organisations is coincidental. Third-party libraries are used under their licences (see app/package.json).